

PARADIGMAS DE PROGRAMAÇÃO

Exame de Preparação Personalizado (Estrutura Reconfigurada)

UC: Paradigmas de Programação (PP)

Ano Letivo: 2025/2026

Duração: 2 horas

Material: Exame sem consulta

Parte I — Aplicação Prática (Conceito do Trabalho)

13,5 valores

1. Construtores, Herança e Validações de Domínio (3,5 valores)

Considere as classes `VehicleImpl` (classe base) e `RefrigeratedVehicleImpl` (subclasse) do teu projeto. Complete a implementação dos construtores abaixo garantindo que:

- O construtor da superclasse `VehicleImpl` lança uma `IllegalArgumentException` se a capacidade for igual ou menor que zero.
- O construtor da subclasse `RefrigeratedVehicleImpl` garante que o veículo apenas é criado se o tipo de bem (`supplyType`) for estritamente do tipo `ItemType.PERISHABLE_FOOD`. Adicionalmente, deve lançar uma exceção se a distância máxima com carga for igual ou inferior a zero.

```
public class VehicleImpl implements Vehicle {
    private final ItemType supplyType;
    private final double maxCapacity;
    private boolean enabled;

    public VehicleImpl(ItemType supplyType, double maxCapacity) {
        // TODO: Implemente o construtor da superclasse
    }
}

public class RefrigeratedVehicleImpl extends VehicleImpl {
    private final double maxDistanceWithCargo;

    public RefrigeratedVehicleImpl(ItemType supplyType, double maxCapacity, double max
        // TODO: Implemente o construtor da subclasse garantindo as validações
    }
}
```

2. Comparabilidade de Objetos em Java: Método equals() (3,5 valores)

No teu projeto, é necessário comparar instâncias de contentores para evitar duplicados. Escreva a implementação do método `equals(Object obj)` para a classe `ContainerImpl` de acordo com as seguintes regras:

- Dois contentores são considerados semanticamente iguais se, e apenas se, possuírem o mesmo código (code) (comparação de strings case-sensitive).
- Garanta que o método respeita todas as boas práticas do contrato clássico do `equals` em Java (proteção contra nulos, reflexividade e compatibilidade de tipos).

```
public class ContainerImpl implements Container {  
    private final String code;  
    private final double capacity;  
  
    public ContainerImpl(String code, double capacity) {  
        this.code = code;  
        this.capacity = capacity;  
    }  
  
    @Override  
    public boolean equals(Object obj) {  
        // TODO: Escreva a sua resposta aqui  
    }  
}
```

3. Gestão Manual de Vetores, Exceções Personalizadas e Resizing (6,5 valores)

Tendo por base o domínio do teu trabalho prático, crie uma classe de gestão chamada `GestaoCaixas` que administre um vetor dinâmico de caixas de suprimentos (`AidBox[]`). **Não é permitida a utilização de qualquer classe do Java Collections Framework (como `ArrayList` ou `HashMap`).**

Implemente os métodos e atributos descritos abaixo, incluindo:

1. O vetor de armazenamento `aidBoxes` e o contador correspondente.
2. O construtor que inicializa o vetor com uma capacidade física inicial de 2 posições.
3. O método `public boolean adicionarCaixa(AidBox caixa)`: adiciona uma caixa ao vetor. Deve:
 - Retornar `false` caso já exista uma caixa no vetor com o mesmo código.
 - Efetuar o ****redimensionamento manual (resizing)**** para o dobro do tamanho caso o vetor esteja cheio.
 - Inserir a caixa e retornar `true`.
4. O método `public double obterCapacidadeTotalPorTipo(ItemType tipo) throws BoxNotFoundException`: calcula e retorna a capacidade máxima total acumulada de todos os contentores daquele `ItemType` instalados na instituição. Se não existirem caixas registadas na instituição, deve lançar a exceção **checked** `BoxNotFoundException`.

```
public class GestaoCaixas {  
    // TODO: Implemente os atributos, construtor, adicionarCaixa e obterCapacidadeTotal  
}
```

Parte II — Conceitos Teóricos e Rastreio

6,5 valores

4. Perguntas Verdadeiro ou Falso (0,5 val. cada) (3,5 valores)

Identifique se as seguintes afirmações são verdadeiras (V) ou falsas (F) e justifique sumariamente.

1. Uma classe declarada como `abstract` em Java não pode, sob qualquer circunstância, conter métodos concretos (com implementação).
2. Em Java, todas as interfaces são implicitamente abstratas, mas os seus métodos podem ser declarados com modificadores de acesso `private` ou `protected` nas novas versões da linguagem.
3. A palavra-chave `super()` pode ser invocada em qualquer linha do corpo de um construtor de uma subclasse, desde que seja executada antes de qualquer atribuição de variáveis.
4. Uma variável de instância de tipo primitivo numérico (como `double capacity`) é automaticamente

inicializada para 0.0 se não for explicitamente inicializada pelo programador.

5. Ao fazer a sobreposição de um método numa subclasse em Java, o método redefinido pode reduzir a visibilidade do método original (ex: redefinir um método `public` como `private`).
6. A palavra-chave `final` aplicada a uma declaração de classe impede que a mesma seja instanciada com o operador `new`.
7. Uma interface em Java pode herdar de múltiplas interfaces através da utilização da palavra-chave `extends`.

5. Conceito de Encapsulamento, Ocultação de Informação e Cópia Defensiva (3,0 valores)

No teu método `getVehicles()` da classe `InstitutionImpl`, foi exigida a clonagem ou reinstituição do vetor de veículos retornado. Explique qual é o propósito desta prática no âmbito do **Encapsulamento** e da **Ocultação de Informação**. O que aconteceria se devolvêssemos a referência direta do vetor privado (`return this.vehicles;`)?